Introduction au Modèle Relationnel et aux Requêtes SQL de Base (Norme ANSI)

Thème principal : Modèle Relationnel, Intégrité Référentielle et Requêtes d'Extraction SQL ISO/ANSI

Synthèse exécutive : Ce cours pose les fondations théoriques et pratiques de la ressource R1.05 (Introduction aux Bases de Données) pour la promotion BUT 1 Informatique. L'objectif pédagogique est de comprendre pourquoi les systèmes informatiques abandonnent les fichiers plats et les tableurs au profit des Systèmes de Gestion de Bases de Données Relationnelles (SGBDR) afin d'éliminer la redondance et de prévenir les anomalies de mise à jour. Le cours formalise la structuration des tables à travers les notions de Clé Primaire (PK) et Clé Étrangère (FK) pour garantir l'intégrité référentielle, puis enseigne la rédaction de requêtes SQL d'extraction d'informations fiables et normalisées selon la norme ISO/ANSI via des jointures explicites INNER JOIN.

1. Du Tableur au Modèle Relationnel : Pourquoi changer de paradigme ?

Dans les applications informatiques réelles, stocker l'information dans un fichier plat (texte, CSV) ou un tableur (Excel) présente des limites majeures d'évolutivité et de fiabilité. Lorsqu'une donnée (comme l'adresse postale d'un client) est répétée sur 50 lignes de commandes différentes, modifier cette adresse exige 50 opérations distinctes. Si une seule ligne est oubliée, la base souffre d'une anomalie de mise à jour et devient incohérente. Le modèle relationnel résout ce problème fondamental à la source : il sépare le système en entités distinctes (par exemple, la table 'Clients' d'un côté et la table 'Commandes' de l'autre) et établit des liaisons logiques rigoureuses entre elles.

- Redondance de données : Duplication d'une même information à plusieurs endroits d'une base de données ou d'un fichier.
- Anomalie de mise à jour : Incohérence provoquée lors de la modification partielle d'une donnée répétée à de multiples reprises.
- SGBD (Système de Gestion de Bases de Données) : Logiciel garantissant la cohérence, la persistance, la sécurité et l'accès concurrentiel aux données.
- Modèle Relationnel : Organisation des données sous forme de tables reliées logiquement par des clés pour éliminer la duplication.

Cas concret enseignant : *Exemple du système bancaire : Si une banque enregistrait le nom, le prénom et l'adresse du titulaire sur chaque ligne de transaction bancaire (virement ou paiement), un simple déménagement exigerait de repasser sur des milliers d'opérations passées. Cela saturerait l'espace de stockage et créerait un risque critique d'erreur de saisie.*

2. Clés Primaires, Clés Étrangères et Intégrité Référentielle

Pour garantir qu'aucun enregistrement ne soit confondu avec un autre dans une table, on définit une clé primaire (PK). Celle-ci possède une contrainte absolue d'unicité et d'existence (elle ne peut pas être nulle). Pour matérialiser un lien entre deux tables (par exemple associer un Étudiant à son Groupe de TD), on n'insère pas toutes les informations du groupe dans la table Étudiant : on se contente d'y stocker la clé primaire du Groupe en tant que Clé Étrangère (FK). Le SGBD contrôle automatiquement toute manipulation sur ces données : il est impossible d'attribuer un numéro de groupe à un étudiant si ce groupe n'existe pas.

- Clé Primaire (Primary Key - PK) : Attribut (ou ensemble d'attributs) identifiant chaque ligne (n-uplet) de façon unique et obligatoirement non nulle (NOT NULL).
- Clé Étrangère (Foreign Key - FK) : Attribut présent dans une table qui fait référence directe à la clé primaire d'une autre table pour matérialiser un lien logique.
- Intégrité Référentielle : Règle métier automatisée par le SGBD interdisant à une clé étrangère de pointer vers un identifiant parent inexistant.

Cas concret enseignant : *Exemple du Numéro de Sécurité Sociale (NIR) ou Identifiant Étudiant : Même si deux étudiants portent exactement les mêmes nom et prénom au Pays Basque (ex: deux Jean Dupont), leur identifiant unique empêche toute collision ou confusion au sein du système d'information de l'IUT.*

3. Extraction de Données en SQL ANSI : SELECT, WHERE et INNER JOIN

Pour extraire des données réparties dans plusieurs tables, la norme moderne SQL ISO/ANSI impose d'utiliser la syntaxe explicite via la clause INNER JOIN associée à la condition de liaison ON (`FROM TableA a INNER JOIN TableB b ON a.fk = b.pk`). Cette forme est nettement supérieure à l'ancienne syntaxe implicite avec virgule (`FROM TableA, TableB WHERE...`) car elle sépare clairement la structure relationnelle (jointure) du filtrage des lignes (clause WHERE). L'utilisation des alias de tables (ex: `Etudiant e`) est une bonne pratique professionnelle indispensable pour simplifier l'écriture et supprimer toute ambiguïté.

- Langage Déclaratif : Le SQL exprime CE QUE l'on cherche et non la méthode algorithmique pour l'obtenir.
- Projection (SELECT) : Spécification explicite des colonnes à extraire et à afficher.
- Filtrage (WHERE) : Application de critères logiques pour cibler uniquement les lignes souhaitées.
- Jointure Explicite (INNER JOIN ... ON) : Fusion logique normalisée ISO/ANSI de deux tables basée sur l'égalité entre Clé Étrangère et Clé Primaire.

Cas concret enseignant : *Mise en garde stricte pour les examens et partiels : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est formellement interdit en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on pas utiliser 'SELECT *' dans un code applicatif ou lors d'un examen ?
Réponse du professeur : La syntaxe 'SELECT *' extrait l'intégralité des colonnes d'une table. En production, cela dégrade fortement les performances réseau et mémoire en transférant des données inutiles. De plus, si la structure de la table évolue (ajout/suppression de colonnes), le code applicatif qui dépend de l'ordre des colonnes risque de planter. En examen, 'SELECT *' ne prouve pas à l'correcteur que vous avez compris quelles colonnes précises étaient demandées.

Question étudiant : Que se passe-t-il si j'essaie de supprimer un groupe dans lequel il y a encore des étudiants rattachés ?
Réponse du professeur : Le SGBD bloque immédiatement l'opération et lève une erreur de contrainte d'intégrité référentielle. Il interdit la suppression d'un enregistrement parent (dans 'Groupe') tant que des enregistrements enfants (dans 'Etudiant') pointent vers sa clé primaire via leur clé étrangère. Pour pouvoir supprimer ce groupe, il faut d'abord réaffecter ou supprimer les étudiants rattachés, ou avoir configuré la contrainte avec une clause 'ON DELETE CASCADE'.

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL conforme à la norme ISO/ANSI permettant d'extraire le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Résolution pas à pas :
Étape 1 : Identifier les tables cibles et leur attribuer des alias clairs -> Etudiant (e) et Groupe (g). Étape 2 : Établir la jointure explicite en reliant la clé étrangère de la table Etudiant à la clé primaire de la table Groupe -> INNER JOIN Groupe g ON e.id_groupe = g.id_groupe. Étape 3 : Appliquer les critères de filtrage dans la clause WHERE -> g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'. Étape 4 : Définir la projection exacte des attributs demandés -> SELECT e.nom, e.prenom.

Résultat : SELECT e.nom, e.prenom FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe WHERE g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'; Commentaire : Cette requête respecte la norme ANSI, évite le produit cartésien et isole parfaitement la condition de jointure du filtre de données.

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux contraintes fondamentales obligatoires que doit respecter une Clé Primaire (Primary Key) ?

R : Une Clé Primaire doit obligatoirement respecter l'unicité stricte (chaque valeur est unique) et l'interdiction des valeurs nulles (contrainte NOT NULL).

Q : Quelle est la différence entre une jointure explicite (INNER JOIN ... ON) et une jointure implicite (produit cartésien dans le FROM) ?

R : La jointure explicite (ANSI) sépare la logique d'association des tables (clause ON) de la logique de filtrage des données (clause WHERE). La jointure implicite mélange tout dans le WHERE après avoir effectué un produit cartésien complet, ce qui est lourd, source d'erreurs d'oubli et pénalisé aux examens.

Q : À quoi sert la contrainte d'intégrité référentielle dans une base de données relationnelle ?

R : Elle garantit qu'une valeur de clé étrangère présente dans une table enfant pointe toujours vers une valeur de clé primaire existante et valide dans la table parent, empêchant ainsi la création d'enregistrements 'orphelins'.